

Курсовой проект
(1-й год обучения; магистратура СПИ)

Описание проекта и заявка на выполнение
(предварительное/доработанное)

Общая информация о проекте*

1. Название работы на русском языке	Архитектура образовательной веб-платформы для совместного программирования с поддержкой CRDT и генерацией учебных подсказок	
2. Название работы на английском языке	Design of a Web-Based Educational Platform for Collaborative Programming with CRDT and Hint Generation	
3. Язык выполнения работы	русский	
4. ФИО студента (автора работы)	Пугач Виктория Павловна, студентка группы МСПИН251	
5. ФИО научного руководителя, его ученое звание, степень и должность	Брейман Александр Давидович, доцент департамента программной инженерии	
6. Подпись научного руководителя и дата подписания	03.02.2026	
7. Подпись студента и дата подписания	03.02.2026	

Содержание

1. Описание предметной области, актуальность и мотивация к ее разработке (исследованию)*	3
1.1. Постановка задачи (problem statement)*	3
1.2 Анализ существующих решений.....	4
1.3 Преимущества предлагаемой системы.....	6
1.4 Педагогический аспект и метрики успешной геймификации.....	7
2. Ключевые слова (keywords).....	10
3. Цель работы (aim of the project)*	10
4. Задачи работы (objectives).....	10
4.1 Функциональные требования	11
4.2 Нефункциональные требования.....	11
4.3 Метрики выполнения поставленных задач. Критерии оценки решения	12
5. Ожидаемые результаты (expected results)*	12
6. Применяемые методы (подходы).....	13
6.1. Архитектура системы	13
6.2. Архитектурные паттерны.....	14
7. Используемые инструменты (технологии)	14
8. Дополнение	15
8.1. Use cases	15
8.2. Планируемый график работы	18
Список источников (references)*	18

1. Описание предметной области, актуальность и мотивация к ее разработке (исследованию)*

Предметной областью данного исследования является проектирование и разработка распределенных коллаборативных платформ для обучения детей программированию в сфере образовательных технологий (EdTech). Работа сосредоточена на решении комплексной задачи, лежащей на стыке двух направлений: создания отзывчивых систем реального времени на основе алгоритмов обеспечения согласованности данных (CRDT) [1] и интеграции интеллектуальных модулей, генерирующих педагогически адаптированные подсказки с учетом когнитивных особенностей детского восприятия. Для управления сложностью данной проблемной области проектирование системы будет вестись с применением принципов предметно-ориентированного проектирования (DDD) [2, 3].

Актуальность работы обусловлена растущим спросом на интерактивные образовательные платформы, которые обеспечивают формирование практических навыков через совместную деятельность, выходя за рамки простой передачи знаний. Однако существующие решения демонстрируют системные ограничения: они либо не обеспечивают технически надежной и бесшовной коллаборации в реальном времени, либо предлагают педагогическую поддержку, не адаптированную для детской аудитории [4, 5]. Подробнее существующие аналоги будут рассмотрены в разделе 1.2. Это приводит к фрагментации учебного процесса, когда для организации групповой работы необходимо комбинировать несколько разнородных инструментов (отдельные редакторы кода, системы видеосвязи, платформы для обмена файлами), ни один из которых не обеспечивает целостного образовательного опыта.

Особую практическую и стратегическую значимость проекту придает текущий контекст доступности IT-инструментов, подтвержденный сотрудничеством с образовательной организацией «Инжиниум» [6]. Блокировка или ограничение работы на российском рынке зарубежных платформ (Roblox, Minecraft) продемонстрировала риски платформозависимости от внешних платформ. Опыт «Инжиниума» показывает, что уход даже узкоспециализированных сервисов (таких как Fusion Brain, использовавшийся для генерации изображений на занятиях) приводит к срыву учебных программ, необходимости срочной переработки методических материалов и поиску аналогов, что критически снижает устойчивость образовательного процесса. Это создает острую потребность в разработке собственных, технологически независимых решений с открытой архитектурой, которые позволяют гибко адаптировать и контролировать всю педагогическую логику.

Таким образом, ключевой проектный вызов заключается в преодолении двойного разрыва: между технологическими возможностями и педагогическими требованиями, а также между зависимостью от внешних сервисов и необходимостью методической стабильности.

Для его решения необходимо обеспечить:

1. Согласованность данных и минимальную задержку при одновременной работе множества пользователей за счет применения CRDT, а также возможность развертывания на собственной инфраструктуре.
2. Создание встроенной системы генерации подсказок, которая напрямую учитывает детскую психологию через упрощение языка, геймификацию и визуальные метафоры.
3. Объединение редактора кода, средств коммуникации и интеллектуальной поддержки в едином пространстве для устранения фрагментации и повышения надежности учебного процесса.

Данный проект направлен не только на создание прототипа образовательной платформы, но и на исследование и проектирование архитектурного решения, которое комплексно интегрирует механизмы CRDT и автономной педагогической поддержки, отвечая на актуальные технологические, образовательные и стратегические вызовы, подтвержденные практикой.

1.1. Постановка задачи (problem statement)*

Ключевая проблема заключается в отсутствии единой, методологически и технологически целостной цифровой среды для проведения онлайн-занятий по программированию для детей. Существующий подход вынуждает образовательные организации создавать экосистему из 4-5

разнородных инструментов: специализированной детской платформы (например, для блочного программирования), профессионального редактора кода или IDE для сложных задач, отдельного сервиса видеоконференций (Zoom, Discord), платформы для сдачи заданий и, зачастую, сторонних API (как в случае с Fusion Brain для генерации контента).

Эта архитектурная фрагментация порождает три взаимосвязанных критических последствия:

1. **Высокий порог отказа и непредсказуемость.** Каждый внешний сервис в цепочке становится точкой отказа. Нестабильность интернет-соединения у ученика, технический сбой или, что наиболее критично, полный уход провайдера с рынка приводит к немедленному срыву занятия или целого учебного модуля. Преподаватель превращается в системного администратора, вынужденного решать технические, а не педагогические задачи.
2. **Низкое качество педагогического взаимодействия и обратной связи.** Постоянное переключение контекстов между приложениями разрушает вовлеченность. Обратная связь для ребенка либо отсутствует, либо представлена техническими сообщениями компилятора, что вызывает фрустрацию. У педагога нет инструментов для наблюдения за групповой работой в реальном времени и корректного вмешательства.
3. **Неисследованность интеграции ключевых технологий.** Существует принципиальный пробел на стыке Computer-Supported Collaborative Learning (CSCL) [7] и Software Engineering. Не изучено, как архитектурно объединить механизм бесшовной синхронизации состояния с модулем семантического анализа и педагогической логики для создания адаптивных подсказок в условиях низкой задержки, характерных для живого урока.

Таким образом, задача данной работы — разработать, обосновать и апробировать архитектурную модель веб-платформы, которая устраняет указанную фрагментацию.

Эта модель должна:

1. Объединить функциональность редактора кода (с поддержкой CRDT), видеосвязи, чата и системы заданий в едином интерфейсе, сокращая обязательные внешние зависимости и повышая отказоустойчивость.
2. Встроить модуль контекстно-зависимой генерации подсказок непосредственно в цикл совместной работы, обеспечивая педагогически релевантную обратную связь на понятном для ребенка языке.
3. Предложить реализуемый паттерн интеграции CRDT-слоя и педагогического движка, что составит конкретный научно-технический вклад в область EdTech.

1.2 Анализ существующих решений

Анализ современных образовательных платформ и профессиональных инструментов выявляет не разрыв, а скорее фрагментацию и недостаточность ключевых возможностей, необходимых для эффективного и непрерывного обучения детей программированию. Три класса решений развивались параллельно, образуя экосистему, где для проведения одного занятия приходится использовать сразу несколько неинтегрированных приложений, а для прогресса ученика — полностью менять инструментарий.

	Code With Me (JetBrains)	Code.org	Scratch	Replit	Tynker/ Blockly	MyWork
Поддержка аудио-/видео-звонков	✓	—	—	—	—	✓
Геймификация процесса обучения	—	✓	—	Частично Бейджи	✓	✓
Наличие готовых курсов	—	✓	✓	✓	✓	✓
Совместное редактирование	✓	Режим работы с 1 девайса (paired) —	Есть пользов. плагины —	✓	Только live demo проектов —	✓
Поддержка блочных ЯП	—	✓	ТОЛЬКО блочные ЯП ✓	—	✓	✓
Поддержка системы оценивания	—	✓	—	—	Встроенной нет —	✓
Адаптивность подсказок	—	✓	✓	—	✓	✓
Адаптивность интерфейса и языка	—	✓	✓	—	✓	✓
Открытость/ Интегрируемость	—	✓	✓	—	—	—**
Поддержка AI	Copilot	—	—	AI Suggestions	—	✓
Целевая аудитория	Профессионалы, Студенты профильных вузов	• Новички • Дети с низким и средним уровнем программирования	• Новички • Дети с низким и средним уровнем программирования	• Студенты • Разработчики • Вайб-кодеры	• Новички • Дети с низким и средним уровнем программирования	• Дети без знания программирования • Продвинутые школьники с навыками программирования
Стоимость	Включено в подписку JetBrains 199+\$/год*	Бесплатно	Бесплатно	Начальный план: 0\$ Поддержка корпоративности, AI, 1200+минут 20-35\$/месяц*	18-54\$/месяц*	** - Закрытая система для заказчика с предусмотренной возможностью расширения в SaaS

* - Оплата российскими картами не поддерживается.

Рисунок 1 – Анализ существующих аналогов

1. Платформы для обучения детей (Scratch, Code.org, Tynker). Эти сервисы обладают сильными педагогическими сторонами: адаптированный для детей интерфейс, встроенная геймификация и структурированные курсы. Однако они критически ограничены технологически: **полностью отсутствует режим реального совместного редактирования кода**. В лучшем случае возможен просмотр общего проекта или поочередная работа. Эти платформы не созданы для синхронной, живой коллаборации, которая лежит в основе современных педагогических методик (peer programming [8], project-based learning [9]). Кроме того, они **имеют чёткий «потолок» сложности** и не предлагают плавного перехода к профессиональным языкам и средам, становясь **непригодными для продвинутых школьников**, освоивших базовые концепции.

2. Профессиональные среды коллаборативной разработки (JetBrains Code With Me, VS Code Live Share, Replit Teams). Эти инструменты решают техническую задачу совместной работы на высоком уровне, предлагая возможность редактирования и, в некоторых случаях, встроенную аудио-/видеосвязь (Code With Me). Однако они абсолютно не адаптированы для детской аудитории. Их интерфейс сложен, сообщения об ошибках носят сугубо технический характер, а отсутствие элементов геймификации и педагогической поддержки делает их непригодными для систематического обучения детей без навыков программирования.

3. Инструменты для коммуникации и демонстрации (Zoom, Discord). Используются для связки первых двух типов, обеспечивая видеосвязь и демонстрацию экрана. Это добавляет еще одно обязательное приложение в рабочий процесс, увеличивая нагрузку на сетевое соединение и вычислительные ресурсы устройств учеников.

Ключевая проблема: Таким образом, для организации полноценного занятия преподаватель и ученики вынуждены одновременно использовать 4-5 разнородных приложений:

1) платформу с заданиями (Scratch), 2) среду для видеосвязи (Zoom), 3) возможно, отдельный редактор кода для сложных задач, 4) мессенджер для оперативной связи.

Эта фрагментация создает следующие барьеры:

1. Технологические. Каждое приложение требует стабильного интернет-соединения. На практике, особенно у детей в удаленных регионах, качество связи нестабильно, и обрыв в любом из сервисов разрушает учебный процесс.
2. Когнитивные. Постоянное переключение между окнами и контекстами увеличивает ментальную нагрузку как на ученика, так и на педагога, снижая эффективность усвоения материала.
3. Педагогические. Отсутствие единого пространства затрудняет проведение быстрых интерактивных активностей, коллективное решение проблем и оперативную выдачу контекстной помощи.

Существует четкая потребность не просто в «новой платформе», а в целостной, интегрированной среде, которая совместит сильные стороны трех указанных классов:

1. Педагогический дизайн и дружелюбие детских платформ.
2. Технологическую надежность совместную работу с минимальными задержками профессиональных инструментов.
3. Встроенную коммуникацию для живого взаимодействия без переключения между приложениями.

Разработка такой среды требует не только интеграции функций, но и принципиально нового **архитектурного подхода**, обеспечивающего стабильность работы в условиях неидеальных сетевых условий, что и составляет ядро данного проекта.

1.3 Преимущества предлагаемой системы

Предлагаемое решение создает единое образовательное пространство, устраняющее ключевой рыночный пробел: отсутствие платформы, которая обеспечивает непрерывный путь развития — от начального блочного программирования к продвинутым текстовым языкам — в технически надежной и мотивирующей среде. Существующие решения образуют разрыв: детские платформы достигают предела сложности и не готовят к реальной разработке, а профессиональные среды лишены педагогической поддержки и геймификации, что демотивирует подростков. Наша платформа синтезирует эти подходы, предлагая бесшовную коллаборацию, адаптивные подсказки и единую траекторию обучения, которая сохраняет вовлеченность ученика по мере роста его навыков.

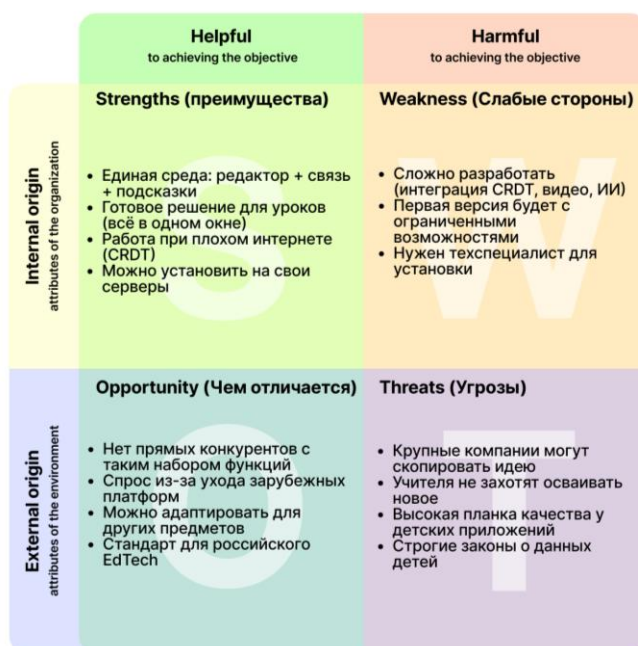


Рисунок 2 – SWOT анализ предлагаемого решения

Для ученика:

1. Непрерывная образовательная траектория: От визуальных блоков к профессиональным языкам в единой среде, что устраняет «потолок» и сохраняет мотивацию продвинутых школьников.
2. Адаптивная и безопасная обратная связь: Контекстные подсказки заменяют пугающие сообщения компилятора, превращая ошибку в шаг обучения. CRDT-алгоритмы обеспечивают работу при нестабильном интернете.
3. Опыт реальной коллаборации: Бесшовное совместное редактирование и встроенные средства коммуникации формируют среду для коллективного решения задач.

Для преподавателя:

4. Единое пространство для ведения занятия: Лекция, групповой мониторинг и индивидуальная помощь происходят в одном окне, снижая организационную нагрузку.
5. Автоматизация рутины: Система подсказок берёт на себя первичный отклик на типовые ошибки, высвобождая время для углубленной работы.
6. Гибкость контента: Модульная архитектура позволяет легко добавлять новые задания и языки под любой уровень подготовки учеников.

Для образовательной организации («Инжиниум»):

7. Технологический суверенитет: Открытая архитектура устраняет риски зависимости от внешних сервисов, обеспечивая надёжность.
8. Методическое развитие: Анализ анонимизированных данных даёт обратную связь для совершенствования программ и построения индивидуальных траекторий.
9. Конкурентное преимущество: Единая платформа, поддерживающая прогресс ученика от новичка до продвинутого уровня, становится ключевым активом.

Система позволяет избежать:

10. Потери мотивации учеников из-за фрагментации инструментов и «потолка» сложности детских платформ.
11. Низкой эффективности групповой работы из-за отсутствия удобных средств для совместного творчества.
12. Профессионального выгорания преподавателей, вынужденных быть «живым интегратором» разрозненных сервисов.

Таким образом, проект создаёт целостную архитектуру для непрерывного обучения, решающую системные проблемы индустрии EdTech.

1.4 Педагогический аспект и метрики успешной геймификации

Данный раздел определяет педагогические принципы, лежащие в основе проектирования системы, и устанавливает ключевые показатели для оценки ее эффективности. Фокус сделан на адаптацию к целевой аудитории — детям 7-17 лет — и измерение не только технической работоспособности, но и реального образовательного и мотивационного воздействия.

1.4.1 Целевая аудитория и принципы возрастной адаптации

Целевая аудитория платформы — дети 7-17 лет, изучающие программирование.

Ключевые возрастные особенности и их влияние на дизайн:

1. Преобладание наглядно-образного мышления. Абстрактные концепции (переменные, циклы) требуют визуализации через метафоры (конвейер, повторяющийся путь). Интерфейс должен быть иконографичным, с минимальным объемом текста.
2. Ограниченный объем произвольного внимания (15-25 минут). Дизайн учебных сессий и заданий должен учитывать этот лимит, разбивая материал на короткие, завершённые этапы с немедленной обратной связью.
3. Высокая эмоциональность, чувствительность к оценке и склонность к отвлечению. Система должна избегать негативных маркеров («Ошибка», «Неверно»), заменяя их на нейтральные или поддерживающие формулировки. Критически важным для удержания внимания является управление видеопотоком. По аналогии с профессиональными сервисами для онлайн-терапии [10], где пациент не видит себя, чтобы сфокусироваться на сессии, в учебной среде дети должны видеть только

преподавателя и, возможно, общий условный аватар класса. Преподаватель должен иметь в интерфейсе возможность отключить для учеников трансляцию их собственной камеры.

Это решает две задачи:

- 1) устраняет распространенный источник балования и отвлечения, когда ребенок играет с изображением себя;
- 2) снижает тревожность и самоконтроль, позволяя ученику полностью сосредоточиться на учебном материале и общении с педагогом. При этом преподаватель сохраняет полный визуальный контроль над классом.
4. Социальная ориентация и потребность в одобрении. Механики геймификации должны поощрять как личные достижения, так и позитивное взаимодействие в группе (совместные награды, похвала за помощь товарищу).

1.4.2 Педагогические метрики и оценка качества

Оценка результатов будет проводиться по двум направлениям: эффективность учебного процесса и качество пользовательского опыта.

1. Метрики эффективности обучения и подсказок:

1. Среднее время решения типовой задачи: Сравнение времени, затраченного на выполнение контрольного задания с использованием системы подсказок и без нее. Ожидается сокращение времени при активации подсказок для новичков.
2. Коэффициент завершения заданий: Процент учащихся, успешно завершивших учебный модуль или задание до конца, в сравнении с контрольной группой, использующей стандартный инструмент.
3. Качество кода. Анализ решений учеников на предмет типичных ошибок. Снижение частоты однотипных ошибок в течение курса будет свидетельствовать об эффективности обратной связи.
4. Оценка понятности подсказок. Анкетирование учащихся после серии заданий с использованием шкалы Ликерта [11] и качественных вопросов. Ключевой критерий — способность ученика самостоятельно сформулировать следующий шаг после получения подсказки.

2. Метрики пользовательского опыта и вовлеченности:

1. Опрос по шкале System Usability Scale [12]. Стандартизированная оценка удобства и простоты использования интерфейса платформы.
3. User Engagement Metrics:
 1. Глубина прохождения. Количество выполненных заданий сверх обязательного минимума.
 2. Использование социальных функций. Количество отправленных интерактивных стикеров-похвал, комментариев в совместных проектах.
 3. Косвенная метрика фокуса внимания. Анализ случаев, когда преподавателю приходится вручную напоминать группе о необходимости вернуться к задаче. Снижение таких инцидентов при использовании режима без самотрансляции будет свидетельствовать об эффективности этого решения.

1.4.3 Признаки успешной геймификации для детей

Успешная геймификация в образовательном контексте — это не просто добавление баллов, а создание внутренней мотивации к процессу обучения.

Ее признаки:

1. Добровольное возвращение к платформе вне учебного времени для работы над личными или дополнительными проектами.
2. Преодоление «страха ошибки». Ученики начинают воспринимать неудачные попытки как часть процесса, а не как провал, что отражается в готовности экспериментировать и запускать код до его идеальной отладки.
3. Социальное взаимодействие вокруг учебного контента. Учащиеся начинают спонтанно обсуждать задания, делиться достижениями или помогать друг другу, используя встроенные инструменты платформы.
4. Фокус на мастерстве, а не на награде. В долгосрочной перспективе удовлетворение учеников смещается от получения внешних знаков отличия к внутреннему ощущению компетентности и понимания.

Педагогический аспект проекта заключается в создании не просто функционального, но психологически безопасного и мотивирующего цифрового пространства, где элементы интерфейса (включая управление видео) напрямую служат учебным целям. Его успех будет измеряться через совокупность объективных метрик выполнения задач и субъективных показателей вовлеченности и удовлетворенности, что обеспечит комплексную валидацию предложенных решений.

1.5 Анализ пользователей и заинтересованных сторон

Роль	Уровень доступа/Влияние	Основные потребности
Ученик (ребенок 7-17 лет)	Основной пользователь. Низкая власть, высокая заинтересованность. Точка влияния: мотивация.	1. Простой, яркий интерфейс, без сложного текста. Визуальный прогресс, анимация. 2. Видеть изменения других в реальном времени без лагов и конфликтов. 3. Ясная, дружелюбная помощь при ошибке, а не текст компилятора.
Преподаватель	Ключевой пользователь. Высокая власть, высокая заинтересованность. Точка влияния: эффективность.	1. Быстрое создание сессии и приглашение учеников (ссылка/код). 2. Обзор активности (кто пишет, типичные ошибки группы), возможность точечного вмешательства. 3. Возможность дать персональную подсказку или «заморозить» код для объяснения.
Администратор (заказчик «Инжиниум»)	Заказчик, владелец. Высокая власть, высокая заинтересованность. Точка влияния: контроль и развитие	1. Стабильность платформы в часы пиковой нагрузки. 2. Сбор анонимизированных метрик (успеваемость, сложность заданий) для улучшения курсов. 3. Возможность добавлять свои задания, языки, развертывать на своих серверах.
Родитель	Косвенный пользователь/спонсор. Низкая власть, высокая заинтересованность. Точка влияния: доверие.	1. Гарантия защиты данных ребенка, соответствие 152-ФЗ. 2. Понятные отчеты об успехах и вовлеченности ребенка. 3. Процесс обучения не должен зависеть от сложных настроек на их стороне.
Технический эксперт (IT-отдел)	Внешний или внутренний специалист. Высокая власть, низкая заинтересованность. Точка влияния: поддерживаемость.	1. Возможность понять, доработать и интегрировать систему. Чистая архитектура. 2. Доступ к данным для мониторинга, отладки и построения своих отчетов. 3. Процедуры развертывания и бэкапа.
Регулятор (Роскомнадзор, закон)	Внешний контекст. Высокая власть, низкая заинтересованность. Точка влияния: соответствие.	1. Строгое соблюдение 152-ФЗ. Законный сбор и хранение персональных данных несовершеннолетних. 2. Критически важно для заказчика, если платформа позиционируется как импортозамещающая, не использовать провайдеры, требующие использования VPN.

Таблица 1 – Основные заинтересованные стороны разрабатываемого решения

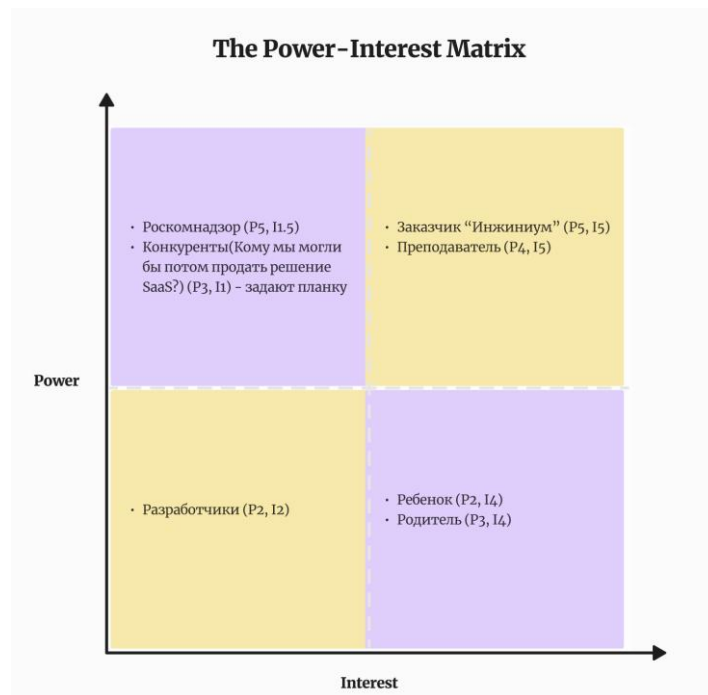


Рисунок 3 – Матрица заинтересованности-влияния заинтересованных сторон [13]

Успех проекта зависит от удовлетворённости трёх ключевых групп с высокой заинтересованностью: Администратора, Преподавателя и Ученика. Требования Родителя и Регулятора формируют необходимые рамки, несоблюдение которых делает проект юридически или рыночно нежизнеспособным. Технические специалисты, заинтересованные в разработке системы, обеспечивают долгосрочную поддержку и эволюцию системы. Параллельно необходимо постоянно анализировать конкурентов, которые не только формируют ожидания пользователей по качеству и функционалу, но и неявно определяют образовательный стандарт, отставание от этой планки может обесценить даже уникальные технологические преимущества платформы.

2. Ключевые слова (keywords)

образовательные технологии; совместное программирование; системы реального времени; CRDT (бесконфликтные реплицируемые типы данных); интеллектуальные обучающие системы; генерация учебных подсказок; программная архитектура; веб-платформа

3. Цель работы (aim of the project)*

Разработать архитектуру и прототип веб-платформы для совместного обучения программированию, которая интегрирует механизм синхронизации на основе CRDT с системой генерации адаптивных учебных подсказок для детей.

Исследовательские вопросы:

1. Каковы паттерны интеграции CRDT-слоя и модуля семантического анализа для генерации контекстных подсказок в real-time среде?
2. Какие метрики объективно оценивают педагогическую эффективность автоматически генерируемых подсказок для детской аудитории?
3. Как обеспечить задержку синхронизации состояния кода менее 100 мс при работе 10+ одновременных пользователей?

4. Задачи работы (objectives)

1. Провести анализ технологий коллаборативного редактирования (CRDT) и методов статического анализа кода для выбора оптимального стека решений. *Срок: 4 недели.*

2. Разработать архитектурную модель системы, включая спецификации API и протоколы взаимодействия модулей. *Срок: 3 недели.*
3. Реализовать прототип коллаборативного редактора кода с базовым механизмом синхронизации на основе CRDT. *Срок: 8 недель.*
4. Интегрировать модуль статического анализа для генерации адаптивных подсказок на типовые ошибки в созданный прототип. *Срок: 4 недели.*
5. Провести нагрузочное тестирование прототипа для подтверждения целевого показателя задержки синхронизации (<100 мс). *Срок: 2 недели.*
6. Подготовить комплект проектной и технической документации. *Срок: 2 недели*

4.1 Функциональные требования

Ядро совместной работы:

1. Система должна предоставлять веб-интерфейс с текстовым редактором кода, поддерживающим подсветку синтаксиса для языков, используемых в обучении детей.
2. Система должна позволять пользователю создавать сессию совместной работы и предоставлять другим пользователям уникальную ссылку для присоединения.
3. Система должна реализовывать механизм синхронизации изменений кода в реальном времени для всех участников сессии с использованием алгоритмов CRDT для гарантированной бесконфликтной конвергенции данных.
4. Система должна отображать список активных пользователей в текущей сессии с индикацией их активности (курсор, выделение текста).
5. Система должна сохранять историю изменений кода в сессии для возможности отката.

Интеллектуальная поддержка и подсказки:

1. Система должна в фоновом режиме выполнять статический анализ набираемого кода.
2. Система должна автоматически обнаруживать набор типовых синтаксических и семантических ошибок, характерных для начинающих программистов.
3. Для обнаруженной ошибки система должна генерировать учебную подсказку, текст которой соответствует принципам адаптации для детского восприятия: краткость, конкретность, использование знакомых понятий, дружелюбный тон, избегание сложной терминологии.
4. Подсказка должна быть визуально привязана к месту ошибки в редакторе.
5. Система должна предоставлять пользователю возможность явно запросить подсказку через интерфейс, если он испытывает затруднения, не связанные с синтаксической ошибкой.
6. Преподаватель должен иметь возможность вручную добавлять текстовые комментарии или подсказки в код для всех участников сессии.

Управление и интерфейс:

1. Система должна предоставлять базовую аутентификацию и регистрацию пользователей.
2. Система должна иметь раздел «Мои проекты/сессии» для доступа к созданным и активным сессиям.
3. Интерфейс должен включать элементы геймификации и позитивного подкрепления: визуальные значки за исправление ошибок, прогресс-бары, похвальные сообщения.
4. Система должна предоставлять RESTful API для интеграции сервиса анализа кода и генерации подсказок с другими системами (в перспективе).

4.2 Нефункциональные требования

1. Производительность. Задержка синхронизации изменений кода должна составлять менее 100 миллисекунд при нагрузке в 10 одновременно активных пользователей в одной сессии.

2. Масштабируемость. Архитектура сервиса реального времени должна быть спроектирована с возможностью горизонтального масштабирования для поддержки роста количества параллельных сессий.
3. Надежность. Клиентское приложение должно обеспечивать офлайн-устойчивость — сохранять локальную копию неотправленных изменений и автоматически восстанавливать состояние сессии после разрыва соединения. Алгоритм CRDT должен гарантировать конечную согласованность данных.
4. Удобство использования. Веб-интерфейс должен быть интуитивно понятен для детей 7-17 лет. Оценка по упрощенному юзабилити-тесту с представителями аудитории должна быть не ниже 70 баллов.
5. Безопасность. Все данные должны передаваться по защищенным протоколам. Доступ к сессии должен контролироваться. Пользовательские данные должны шифроваться при хранении.
6. Интегрируемость. Архитектура должна быть модульной, четко разделяя компоненты совместного редактирования, анализа кода и бизнес-логики для упрощения дальнейшего развития (например, интеграции AI-моделей).
7. Архитектурная четкость. Архитектура системы должна быть спроектирована с выделением ограниченных контекстов в соответствии с принципами DDD для разделения доменов совместного редактирования, педагогического анализа и управления обучением.

4.3 Метрики выполнения поставленных задач. Критерии оценки решения

Категория	Ключевые критерии и целевые показатели	Методы верификации
Функциональная полнота	• Реализованы все 15 функциональных требований (раздел 3.1). • Обеспечена бесконфликтная синхронизация правок.	1. Ручное тестирование по чек-листу сценариев. 2. Приемочное тестирование с представителем заказчика.
Техническая эффективность (Бесперебойность)	• Задержка синхронизации: < 100 мс при 10 пользователях в сессии. • Время генерации подсказки: < 2 секунды. • Надежность: Восстановление сессии после разрыва соединения без потери данных.	1. Нагрузочное тестирование инструментами. 2. Тестирование отказоустойчивости (имитация обрыва сети).
Педагогическая адекватность	• Понятность подсказок: ≥ 80% по оценке методиста. • Удобство интерфейса для детей: ≥ 70 баллов по шкале SUS. • Общая удовлетворенность: средняя оценка ≥ 4.0 из 5 от учеников.	1. Экспертная оценка методистом «Инжиниум». 2. Юзабилити-тестирование с детьми (фокус-группа). 3. Анкетирование после пилотного занятия.

5. Ожидаемые результаты (expected results)*

1. Документированная архитектура системы, утвержденная научным руководителем и представителем заказчика («Инжиниум»).
2. Функционирующий прототип, демонстрирующий:
 - модуль совместного редактирования с базовой реализацией CRDT;
 - сервис анализа кода и генерации простых подсказок;
 - веб-интерфейс для проведения сессии.
3. Технические метрики прототипа (KPI):

- задержка синхронизации (от действия до отражения): < 100 мс при 10 пользователях;
 - время генерации подсказки: < 2 секунд.
4. Экспертная оценка: релевантность и понятность генерируемых подсказок признана удовлетворительной педагогом-экспертом от «Инжиниум».

6. Применяемые методы (подходы)

1. Проектирование архитектуры (UML). Используется для формализации и наглядного отображения модульной структуры (диаграммы классов, компонентов) и потоков данных, соответствующих DDD. Проще для статической документации, чем альтернативы (C4, ArchiMate).
2. Предметно-ориентированное проектирование (DDD). Критичен для управления сложностью через выделение ограниченных контекстов: "Совместное редактирование", "Педагогический анализ", "Управление сессиями". Альтернативная многослойная архитектура привела бы к сильному зацеплению модулей.
3. Алгоритмы CRDT. Выбран для обеспечения офлайн-устойчивости и бесконфликтного слияния правок. Альтернатива OT отвергнута, так как требует центрального сервера, создавая точку отказа и задержку, что нарушает НФТ по надежности (<100 мс).
4. Статический анализ кода (на первом этапе создания прототипа). Основа для генерации контекстных подсказок.
Альтернативы:
 - *Regex*: слишком примитивен, не понимает структуру кода.
 - *Внешний AI (LLM)*: имеет непредсказуемую задержку, требует сети. AST-анализ локальный, быстрый (<2 сек), детерминированный и соответствует требованиям для типовых ошибок (ФТ 7,8). Однако в дальнейшем рассматривается возможность создания сервиса интеллектуальной генерации подсказок при помощи LLM.
5. Нагрузочное тестирование. Необходимо для верификации НФТ по задержке (<100 мс). Используется сценарный подход с инструментами для интеграции в CI/CD.

6.1. Архитектура системы

Выбрана модульная монолитная архитектура с разделением на ограниченные контексты (предметно-ориентированное проектирование, DDD) и использованием шаблона "Порты и адаптеры".

Обоснование выбора:

1. Соответствие НФТ по задержке (<100 мс). Взаимодействие между модулями происходит через вызовы в памяти, что исключает сетевые задержки, неизбежные в микросервисной архитектуре.
2. Соответствие НФТ по архитектурной четкости. Позволяет выделить три ключевых контекста с минимальным зацеплением:
 - "Совместное редактирование" (синхронизация CRDT)
 - "Педагогический анализ" (генерация подсказок)
 - "Управление обучением" (сессии, пользователи)
3. Оптимальность для масштаба проекта. Балансирует между простотой разработки монолита и удобством поддержки за счет строгой модульности, что эффективно для команды из одного разработчика.

Рассмотренные и отвергнутые альтернативы:

- Классический монолит отвергнут из-за сильного зацепления кода, что нарушает требования по интеграции.
- Микросервисная архитектура отвергнута на этапе прототипа из-за операционной сложности (оркестрация, мониторинг) и риска невыполнения НФТ по задержке.

Структурная схема:

1. Модуль collaboration-core включает CRDT-логику и WebSocket-шлюз.

2. Модуль hint-engine включает анализатор кода и правило-движок генерации подсказок.
3. Модуль session-manager управляет сессиями, аутентификацией и видеосвязью.
4. API-шлюз (API Gateway) объединяет REST-маршруты и проксирует WebSocket-соединения.
5. Клиентское одностраничное приложение (SPA) подключается к API-шлюзу и модулю синхронизации для минимальной задержки.

Основное преимущество схемы — соблюдение всех нефункциональных требований при минимальных операционных издержках на этапе разработки прототипа.

6.2. Архитектурные паттерны

1. Шаблон «Порты и адаптеры» (Hexagonal Architecture)
 - Контекст: Взаимодействие модулей и интеграция с внешними системами.
 - Реализация: Каждый модуль предоставляет порты (интерфейсы). Другие модули подключаются к ним через адаптеры, что изолирует внутренние изменения.
 - Цель: Поддержка НФТ по интегрируемости и архитектурной четкости, подготовка к возможной эволюции в микросервисы.
2. Шаблон «Наблюдатель» (Observer)
 - Контекст: Рассылка CRDT-операций клиентам и уведомление подписчиков (модуля подсказок) об изменении кода.
 - Реализация: Модуль collaboration-core публикует событие CodeUpdated, на которое подписан модуль hint-engine.
 - Цель: Асинхронная и слабосвязанная интеграция ядра синхронизации и движка подсказок, что не блокирует обработку операций и соответствует требованию низкой задержки.
3. Шаблон «Единый источник истины» (Single Source of Truth)
 - Контекст: Управление состоянием сессии в модуле session-manager.
 - Реализация: Состояние сессии хранится в централизованном хранилище (Redis). Все изменения выполняются как транзакционные операции над ним.
 - Цель: Гарантия консистентности данных о сессии для всех компонентов, что обеспечивает надежность (НФТ 3).
4. Шаблон «Шлюз API» (API Gateway)
 - Контекст: Единая точка входа для клиентского приложения.
 - Реализация: Компонент (на базе Express.js), который маршрутизирует HTTP-запросы к модулям и проксирует WebSocket-соединения.
 - Цель: Инкапсуляция внутренней структуры, упрощение клиентского кода, централизация аутентификации и логирования, что важно для безопасности (НФТ 5) и масштабирования.

7. Используемые инструменты (технологии)

Выбор технологий сбалансирован между скоростью разработки, соответствием НФТ и поддержкой архитектурной чистоты.

- Фронтенд: React, TypeScript
- Бэкенд: Node.js, Express, [Socket.IO](#)
- Базы данных: PostgreSQL, Redis
- CRDT-библиотека: Yjs
- Анализ кода: Babel Parser (для JS), ast (для Python)
- Контейнеризация: Docker, Docker Compose
- Документация API: OpenAPI (Swagger)

React выбран зрелая экосистема для real-time с TypeScript для строгой типизации данных CRDT и API-контрактов. Альтернативы (Vue, Svelte) менее подходят.

Для бэкенда выбран Node.js для единой JS-экосистемы и высокой конкурентности I/O-операций. [Socket.IO](#) поверх WebSocket обеспечивает надежные сессии.

В качестве БД выбран PostgreSQL для ACID-транзакций и структурированных данных (пользователи, проекты). Redis — для состояния сессий, кэша и pub/sub.

В качестве CRDT-библиотеки Yjs выбрана за производительность, встроенную поддержку WebSocket и зрелые интеграции с редакторами (альтернативы: Automerge, Lexical).

Для генерации подсказок используется статический анализ AST (на базе Babel Parser для JavaScript). Более сложный LLM-подход рассматривается для будущих версий.

OpenAPI (Swagger) выбран для автоматической генерации спецификаций, что критично для интегрируемости (НФТ 6) и будущего перехода к SaaS-модели.

Docker и Docker Compose выбран для локальной разработки; для production рассматривается оркестрация (Kubernetes) для горизонтального масштабирования.

8. Дополнение

Разработка ведется в тесном взаимодействии с заказчиком — образовательной организацией «Инжиниум».

План внедрения включает:

1. Фаза Alpha-тестирования. Внутренний прогон ключевых сценариев преподавателями «Инжиниум» для оценки педагогической логики и интерфейса.
2. Фаза Beta-тестирования. Пилотное занятие с группой учеников (5-10 человек) для сбора метрик по задержке, понятности подсказок и вовлеченности (SUS-опрос).
3. Критерий перехода. Успешное прохождение тестирования и положительная экспертная оценка методиста «Инжиниум». Это обеспечивает не только практическую значимость, но и валидность результатов исследования.

Проект рассматривает два сценария, влияющих на технические:

1. Сценарий А (B2B для образовательных организаций): Поставка платформы как коробочного решения с самостоятельным развертыванием у заказчика. Требует качественной документации и инструментов для миграции данных.
2. Сценарий В (SaaS): Предоставление платформы как облачного сервиса. Документированное OpenAPI-описание ключевых сервисов (hint-engine, session-manager) является первым шагом к созданию публичного API, что откроет возможность для создания маркетплейса плагинов и интеграций третьими сторонами.

8.1. Use cases

На основе анализа ключевых заинтересованных лиц были выделены две фундаментальные потребности, формирующие уникальное ценностное предложение платформы. Для их детальной проработки и формализации сформулированы следующие ключевые варианты использования (Use Cases):

1. «Совместное редактирование кода в сессии». Данный сценарий напрямую отвечает на потребность в бесперебойной и бесконфликтной групповой работе, особенно в условиях нестабильного сетевого соединения. Он служит прямым обоснованием для ядра технических требований: ФТ-3, ФТ-4 (CRDT-синхронизация, индикация активности) и критических НФТ-1, НФТ-3 (задержка <100 мс, надежность и офлайн-устойчивость).
2. «Получение подсказки при ошибке». Этот сценарий формализует потребность в мгновенной и педагогически корректной обратной связи, снижающей когнитивную нагрузку на ученика и преподавателя. Он является основой для детальных требований к интеллектуальному модулю системы: ФТ-6, ФТ-7, ФТ-8, ФТ-9, ФТ-10 (анализ AST, генерация адаптированных подсказок). Его успешная реализация — главный критерий для оценки педагогической адекватности платформы.

Название	Совместное редактирование кода в сессии
Рамки применения	Система совместного программирования
Значимость	Ключевая задача
Первичный актер	Ученик (ребенок 6-17 лет)

Вторичные актеры	Сервер синхронизации (обрабатывает CRDT-операции, обеспечивает рассылку)
Базовый поток	A. Ученик редактирует код. B. Система отправляет CRDT-операцию на сервер. C. Сервер рассылает операцию всем участникам. D. Система каждого ученика применяет операцию. E. Код у всех отображается идентично.
Альтернативные потоки	A1. Восстановление при разрыве связи: A1-A. Потеря соединения → буферизация изменений. A1-B. Восстановление → отправка операций. A1-C. CRDT обеспечивает согласованность. D1. Конфликт операций: Автоматическое разрешение CRDT. D2. Длительный разрыв (>5 мин): Предложение переподключиться.
Предусловия	Ученик авторизован и находится в активной сессии совместной работы.
Постусловия	Код у всех участников сессии синхронизирован.

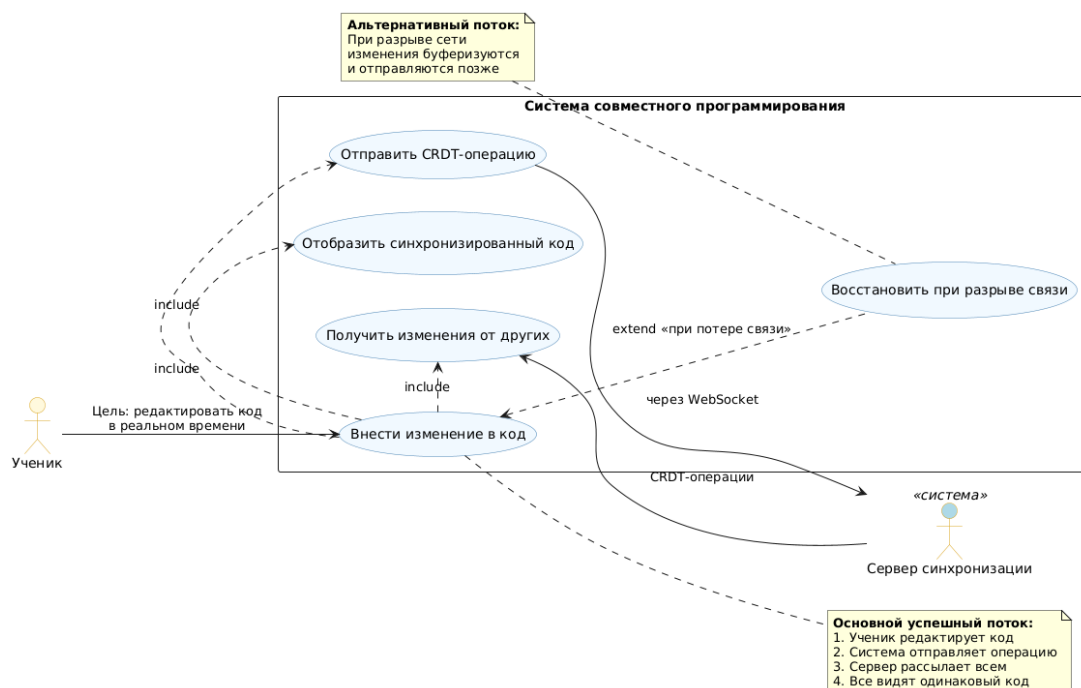


Рисунок 4 – Use Case: Совместное редактирование кода в сессии

Название	Получение подсказки при ошибке
Рамки применения	Система интеллектуальной поддержки обучения
Значимость	Ключевая задача (педагогическая составляющая)
Первичный актер	Ученик / Система
Вторичные актеры	Сервис анализа кода (статический анализ, AST, генерация подсказок)

Базовый поток	A. Система обнаруживает ошибку в коде. B. Код отправляется в сервис анализа. C. Сервис генерирует адаптивную подсказку. D. Подсказка отображается с подсветкой места ошибки. E. Ученик получает понятное объяснение.
Альтернативные потоки	A1. Ручной запрос: Ученик нажимает "Помощь" → анализ → подсказка по логике задачи. C1. Сервис недоступен: Сообщение "Сервис подсказок временно недоступен". C2. Неопределенная ошибка: Общая подсказка "Проверь символы в строке". D1. Частые запросы: Временное ограничение с советом подумать самостоятельно.
Предусловия	Ученик работает в редакторе. Для автоматического потока — есть ошибка в коде.
Постусловия	Ученик получает адаптированную подсказку. Ошибка становится понятной/исправляется.

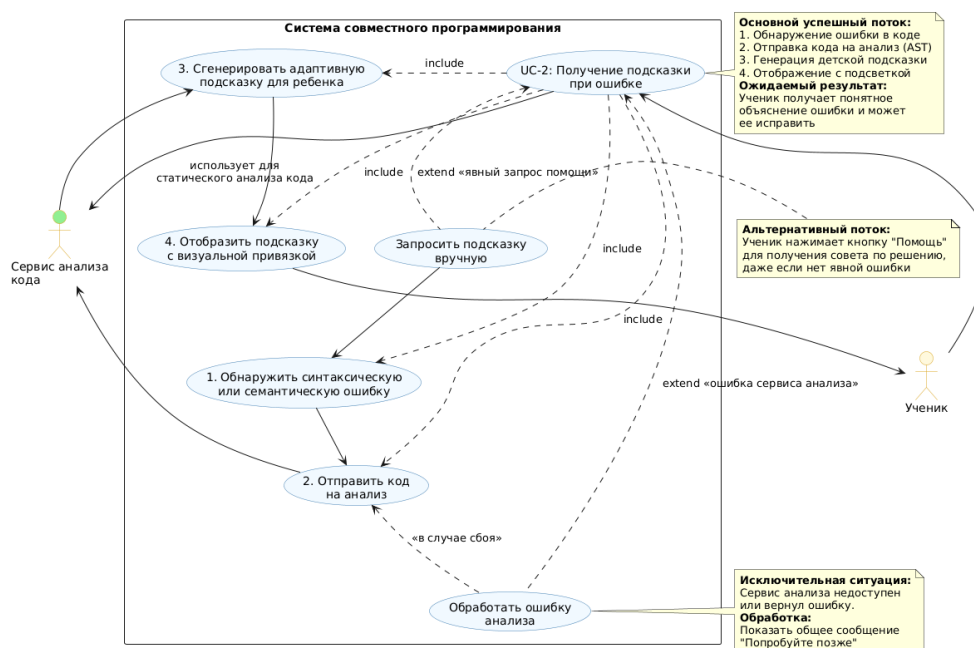


Рисунок 5 – Use Case: Получение подсказки при ошибке

Представленные Use Cases фокусируются на двух наиболее сложных и инновационных аспектах проекта, которые и составляют его научно-техническую новизну. Остальные функциональные требования являются поддерживающей инфраструктурой, необходимой для реализации этих ключевых сценариев.

8.2. Планируемый график работы

График построен на основе задач из раздела 4. Ключевые вехи (Milestones) обозначены как M1-M4.



Рисунок 6 – Гант-диаграмма плана разработки предлагаемого прототипа

Список источников (references)*

1. Shapiro, M. Conflict-Free Replicated Data Types [Текст] / M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski // Lecture Notes in Computer Science. — 2011. — P. 386–400. — DOI: [10.1007/978-3-642-24550-3_29](https://doi.org/10.1007/978-3-642-24550-3_29).
2. Delev, T. Static analysis of source code written by novice programmers [Текст] / T. Delev, D. Gjorgjevikj // 2017 IEEE Global Engineering Education Conference (EDUCON). — 2017. — P. 825–830. — DOI: [10.1109/EDUCON.2017.7942942](https://doi.org/10.1109/EDUCON.2017.7942942).
3. Crow, T. Intelligent tutoring systems for programming education [Текст] / T. Crow, A. Luxton-Reilly, B. Wuensche // Proceedings of the 20th Australasian Computing Education Conference (ACE '18). — 2018. — DOI: [10.1145/3160489.3160492](https://doi.org/10.1145/3160489.3160492).
4. Vernon, V. Domain-Driven Design Distilled [Электронный ресурс] / V. Vernon. — URL: <https://cdn.bookee.app/files/pdf/book/en/domain-driven-design-distilled.pdf> (дата обращения: 03.02.2026).
5. Эванс, Э. Предметно-ориентированное проектирование (DDD): структуризация сложных программных систем [Текст] / Э. Эванс. — Москва: Вильямс, 2011. — 448 с.
6. Инжиниум: собственная экосистема обучения [Электронный ресурс] // Проектная ИТ-школа «Инжиниум». — URL: <https://inginium.ru/> (дата обращения: 03.02.2026).
7. Computer-supported collaborative learning [Электронный ресурс] // Wikipedia. — 2025. — URL: https://en.wikipedia.org/wiki/Computer-supported_collaborative_learning (дата обращения: 03.02.2026).
8. Pair programming [Электронный ресурс] // Wikipedia. — 2020. — URL: https://en.wikipedia.org/wiki/Pair_programming (дата обращения: 03.02.2026).
9. Project-based learning [Электронный ресурс] // Wikipedia. — 2019. — URL: https://en.wikipedia.org/wiki/Project-based_learning (дата обращения: 03.02.2026).
10. Психологи онлайн на Ясно [Электронный ресурс] // yasno.live. — URL: <https://yasno.live/> (дата обращения: 03.02.2026).
11. Шкала Лайкерта [Электронный ресурс] // Википедия. — 2009. — URL: https://ru.wikipedia.org/wiki/Шкала_Лайкерта (дата обращения: 03.02.2026).
12. System usability scale [Электронный ресурс] // Wikipedia. — 2019. — URL: https://en.wikipedia.org/wiki/System_usability_scale (дата обращения: 03.02.2026).
13. A Guide to the Project Management Body of Knowledge (PMBOK® Guide) [Текст] / Project Management Institute. — 6th ed. — 2017.

Для редактирования текста и проверки на наличие лексических и грамматических ошибок использованы следующие языковые модели:

14. ChatGPT (версия от 14 марта 2023) [Большая языковая модель] / OpenAI. — 2023. — URL: <https://chat.openai.com/> (дата обращения: 03.02.2026).
15. DeepSeek Chat [Большая языковая модель] / DeepSeek. — URL: <https://chat.deepseek.com/> (дата обращения: 03.02.2026).